```
  it "can raise an error" do
    allow(test_double).to receive(:message) do
      # TODO
    end

    expect { test_double.message }.to raise_error(/boom/)
  end

  it "can yield a value" do
    allow(test_double).to receive(:message) do |&block|
      # TODO
    end

    expect { |b| test_double.message(&b) }.to yield_with_args(1)
  end

  it "can throw a symbol" do
    allow(test_double).to receive(:message) do
      # TODO
    end

    expect { test_double.message }.to throw_symbol(:foo)
  end
end
```

The final three examples deal with constraining method calls via a block:

14-customizing-test-doubles/exercises/block_implementation_spec.rb

```
RSpec.describe "Block implementations that check calls" do
  let(:test_double) { double }

  it "can constrain arguments" do
    allow(test_double).to receive(:message) do |arg|
      # TODO
    end

    expect { test_double.message(:valid_arg) }.not_to raise_error
    expect { test_double.message(:invalid_arg) }.to raise_error(/invalid_arg/)
  end

  it "can count how many times the message was received" do
    receive_count = 0

    allow(test_double).to receive(:message) do |&block|
      # TODO
    end

    test_double.message
    test_double.message

    expect(receive_count).to eq(2)
  end
```